

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Navya Maheshwari – 24011101071
AI & DS – B1

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output. The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum:

$$z = \mathbf{w}^T \mathbf{x} + b$$

where $\mathbf{x}$ is the input vector, $\mathbf{w}$ the weight vector, b the bias, and z the linear combination.

Prediction:

$$\hat{y} = f(z)$$

Weight Update Rule:

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update:

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function.

4. Dataset Description

Dataset: Banknote Authentication Dataset **Source:** UCI Machine Learning Repository
<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Features: Variance, Skewness, Curtosis, Entropy. Target class: 0 – Authentic Banknote, 1 – Forged Banknote.

5. Experimental Procedure

Task 1: Dataset Exploration — Load the dataset, display the first five samples, determine dimensions, identify missing values, and display descriptive statistics.

Task 2: Exploratory Data Analysis — Generate histograms, correlation heatmap, scatter plots, and boxplots.

Task 3: Data Preprocessing — Normalize all numerical features and split into 80% training / 20% testing.

Task 4: Perceptron Implementation — Implement from scratch: weight initialization, bias initialization, step activation, forward propagation, and the perceptron learning rule.

Task 5: Model Training — Display epoch number, misclassified samples, updated weights, and updated bias.

Task 6: Model Evaluation — Compute accuracy, precision, recall, F1-score, and the confusion matrix.

6. Source Code

<https://github.com/nzoopxw/deep-learning-lab-sem5/tree/main/Lab-01>

7. Experimental Results

Dataset Overview

The dataset has dimensions 1372×5 (4 numerical features and 1 target), with no missing values. The class distribution is 762 authentic (0) and 610 forged (1).

Descriptive Statistics

Metric	Variance	Skewness	Curtosis	Entropy	Class
Count	1372	1372	1372	1372	1372
Mean	0.4337	1.9224	1.3976	-1.1917	0.4446
Std	2.8428	5.8690	4.3100	2.1010	0.4971
Min	-7.0421	-13.7731	-5.2861	-8.5482	0.0000
25%	-1.7730	-1.7082	-1.5750	-2.4135	0.0000
50%	0.4962	2.3197	0.6166	-0.5867	0.0000
75%	2.8215	6.8146	3.1793	0.3948	1.0000
Max	6.8248	12.9516	17.9274	2.4495	1.0000

Task-wise Outputs

Task 1: Dataset Exploration. The dataset was loaded and its first five samples, dimensions, missing-value counts, and descriptive statistics were printed.

```

First five samples:
  Variance  Skewness  Curtosis  Entropy  Class
0   3.62160    8.6661  -2.8073 -0.44699    0
1   4.54590    8.1674  -2.4586 -1.46210    0
2   3.86600   -2.6383   1.9242  0.10645    0
3   3.45660    9.5228  -4.0112 -3.59440    0
4   0.32924   -4.4552   4.5718 -0.98880    0

Dataset dimensions: (1372, 5)
Instances: 1372 | Columns: 5

Missing values per column:
Variance    0
Skewness    0
Curtosis    0
Entropy     0
Class       0
dtype: int64

Descriptive statistics:
      Variance    Skewness    Curtosis    Entropy    Class
count  1372.000000  1372.000000  1372.000000  1372.000000  1372.000000
mean     0.433735    1.922353    1.397627   -1.191657    0.444606
std      2.842763    5.869047    4.310030    2.101013    0.497103
min     -7.042100   -13.773100   -5.286100   -8.548200    0.000000
...
Class
0      762
1      610
Name: count, dtype: int64
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Figure 1: Dataset summary and statistical information.

Task 3: Data Preprocessing. Features were standardized (zero mean, unit variance) and split 80/20 into training and testing sets.

```

Before normalization (first row): [ 3.6216  8.6661 -2.8073 -0.44699]
After normalization (first row): [ 1.1218  1.1495 -0.976  0.3546]
Feature means after scaling: [ 0.  0.  0. -0.]
Feature stds after scaling : [1.  1.  1.  1.]

Training set: (1097, 4) | Testing set: (275, 4)
Train class balance: [609 488]
Test class balance : [153 122]

```

Figure 2: Normalized feature values and train/test split shapes.

Task 5: Model Training. The perceptron was trained for 50 epochs, printing the misclassified count, updated weights, and bias after each epoch.

Epoch	Misclassified	Weights	Bias
1	59	[-0.068 -0.0938 -0.0678 -0.011]	-0.0300
2	25	[-0.0775 -0.1042 -0.0811 0.0035]	-0.0400
3	20	[-0.0802 -0.1127 -0.0917 -0.0036]	-0.0400
4	19	[-0.0883 -0.1196 -0.1001 0.0089]	-0.0500
5	20	[-0.0991 -0.1295 -0.1037 0.0102]	-0.0500
6	17	[-0.1027 -0.1277 -0.1134 -0.001]	-0.0600
7	24	[-0.1081 -0.1393 -0.1108 -0.0094]	-0.0600
8	22	[-0.117 -0.1357 -0.1319 -0.0025]	-0.0600
9	20	[-0.1176 -0.147 -0.1364 -0.0062]	-0.0600
10	20	[-0.1307 -0.1541 -0.125 -0.006]	-0.0600
11	22	[-0.1345 -0.1641 -0.1329 -0.0018]	-0.0600
12	20	[-0.1287 -0.16 -0.1472 0.0076]	-0.0800
13	16	[-0.1343 -0.1627 -0.1458 0.0039]	-0.0800
14	24	[-1.357e-01 -1.673e-01 -1.549e-01 1.000e-04]	-0.0800
15	17	[-0.1362 -0.1802 -0.1518 -0.015]	-0.0700
16	21	[-0.1464 -0.1877 -0.1507 -0.0034]	-0.0600
17	19	[-0.143 -0.1793 -0.1685 -0.0169]	-0.0700
18	18	[-0.1603 -0.1845 -0.1543 -0.0013]	-0.0700
19	23	[-0.1503 -0.1853 -0.1712 -0.0101]	-0.0800
20	19	[-0.1636 -0.1946 -0.1613 -0.0064]	-0.0700
21	18	[-0.1485 -0.1985 -0.1725 -0.0064]	-0.0900
22	15	[-0.1575 -0.1939 -0.1715 -0.0254]	-0.0800
23	12	[-0.1599 -0.1909 -0.1749 -0.0174]	-0.0800
...			
47	16	[-0.1987 -0.2279 -0.218 -0.0179]	-0.0900
48	17	[-0.1951 -0.2336 -0.2113 -0.0119]	-0.1000
49	14	[-0.2092 -0.2231 -0.2069 -0.0255]	-0.1000
50	18	[-0.1941 -0.2358 -0.2109 -0.0195]	-0.1000

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Figure 3: Epoch-wise training log (epoch, misclassified samples, weights, bias).

Task 6: Model Evaluation. The trained model was evaluated on the test set.

```
===== PERCEPTRON EVALUATION (Test Set) =====
Accuracy : 0.9855
Precision: 0.9683
Recall   : 1.0000
F1-score : 0.9839

Classification Report:

```

	precision	recall	f1-score	support
Authentic (0)	1.0000	0.9739	0.9868	153
Forged (1)	0.9683	1.0000	0.9839	122
accuracy			0.9855	275
macro avg	0.9841	0.9869	0.9853	275
weighted avg	0.9859	0.9855	0.9855	275

Figure 4: Accuracy, precision, recall, F1-score, and classification report.

8. Generated Plots with Interpretation

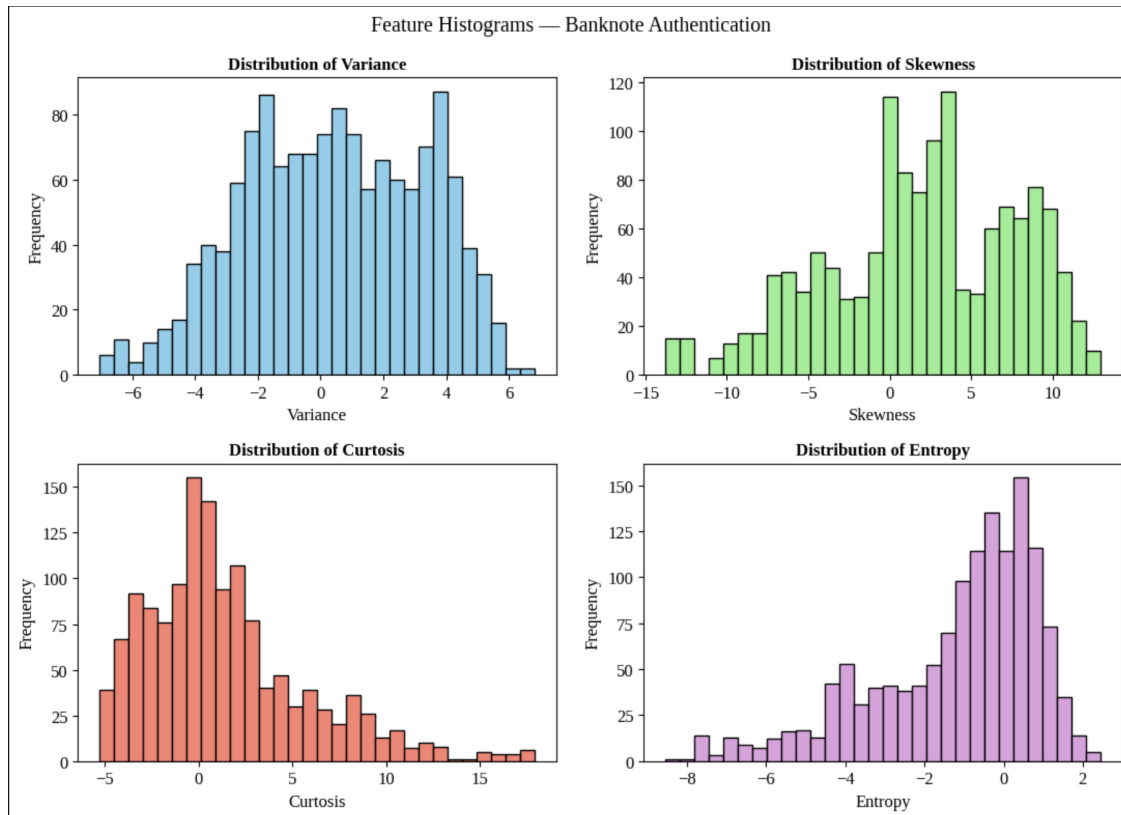


Figure 5: Feature Histograms.

Interpretation: Variance and Skewness show broad, near-symmetric spreads, suggesting they carry meaningful separation between classes. Kurtosis is right-skewed with a sharp peak near zero and Entropy is left-skewed, indicating these two features are less evenly distributed and likely contribute less standalone discriminative value.

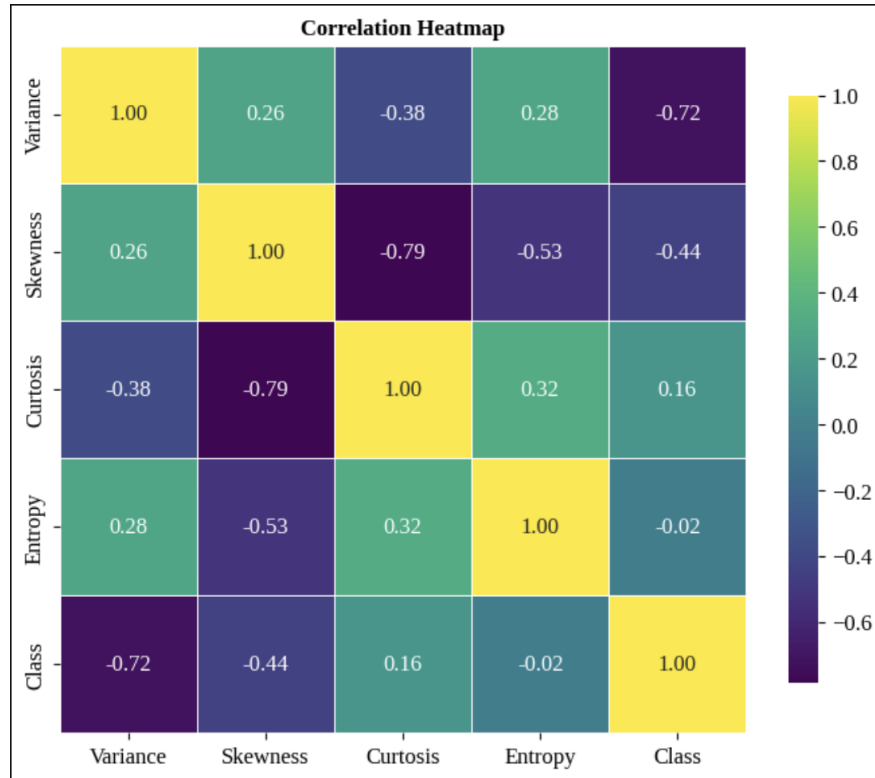


Figure 6: Correlation Heatmap.

Interpretation: Variance has the strongest correlation with the target, making it the most predictive single feature, followed by Skewness. Curtosis and Entropy correlate weakly with the target, so they add little direct linear predictive value on their own. Curtosis and Skewness are strongly negatively correlated with each other, indicating some redundancy.

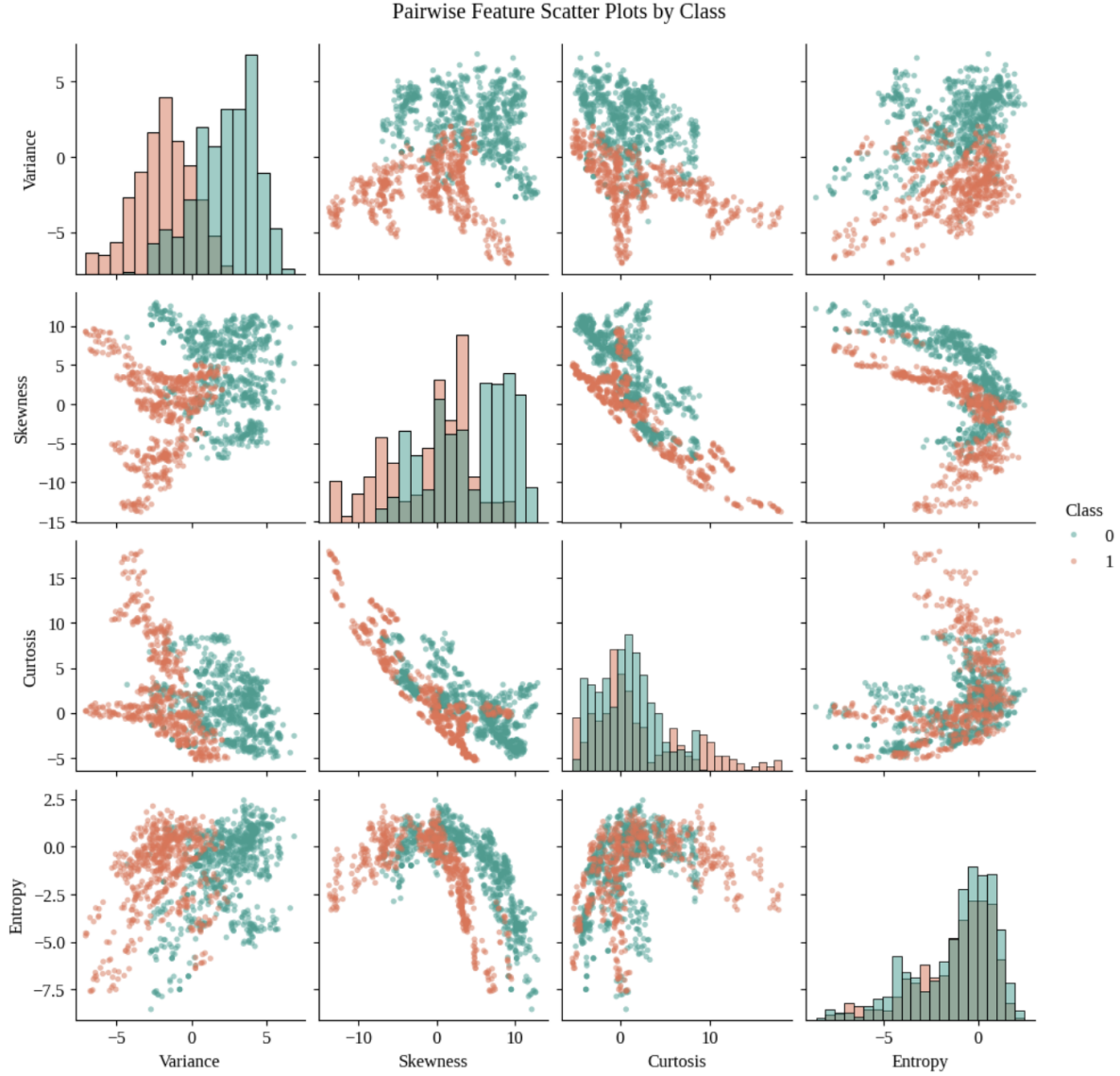


Figure 7: Pairwise Scatter Plots by Class.

Interpretation: Pairs involving Variance show the clearest visual separation between the two classes, confirming Variance as the dominant discriminating feature. Pairs of the weaker features (Skewness vs Entropy, Kurtosis vs Entropy) show heavy overlap. Overall the classes are approximately, though not perfectly, linearly separable.

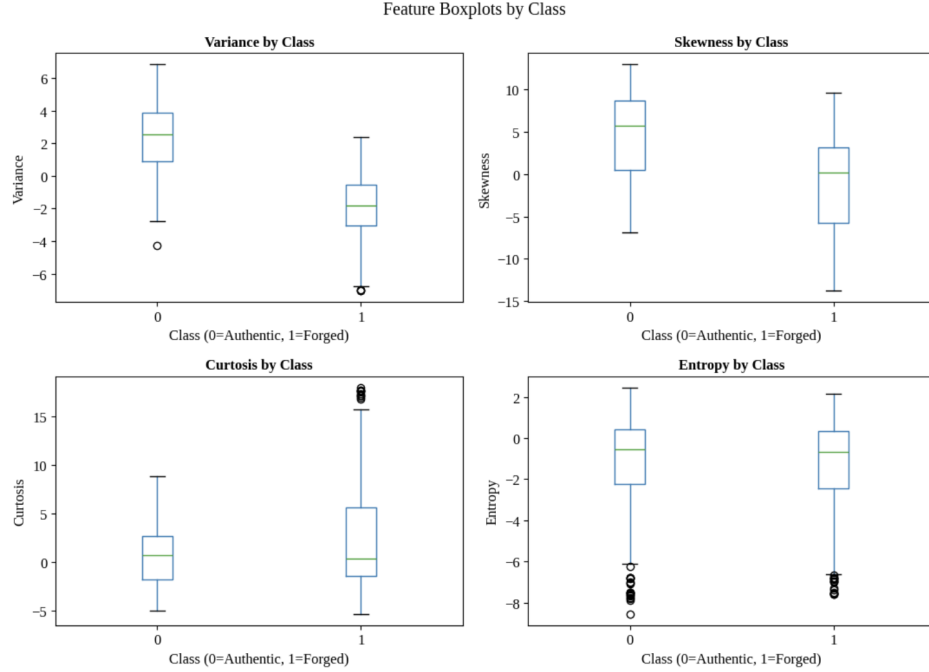


Figure 8: Feature Boxplots by Class.

Interpretation: Variance and Entropy have relatively tight interquartile ranges, while Skewness shows the widest spread. Kurtosis and Entropy display a large number of outliers, indicating these two features are noisier and less reliable as clean standalone predictors.

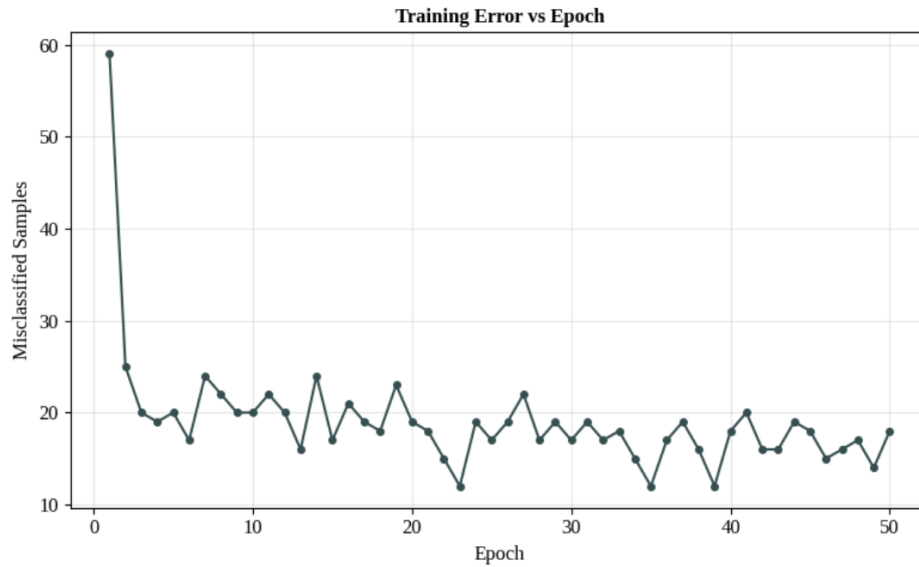


Figure 9: Training Error vs Epoch.

Interpretation: The misclassification count drops sharply in the first few epochs (from 59 to around 20) and then plateaus in the range of roughly 12–25 for the remainder of training. It does not reach zero, indicating the Banknote classes are nearly but not perfectly linearly separable; nonetheless the error stabilizes at a low level, showing effective learning.

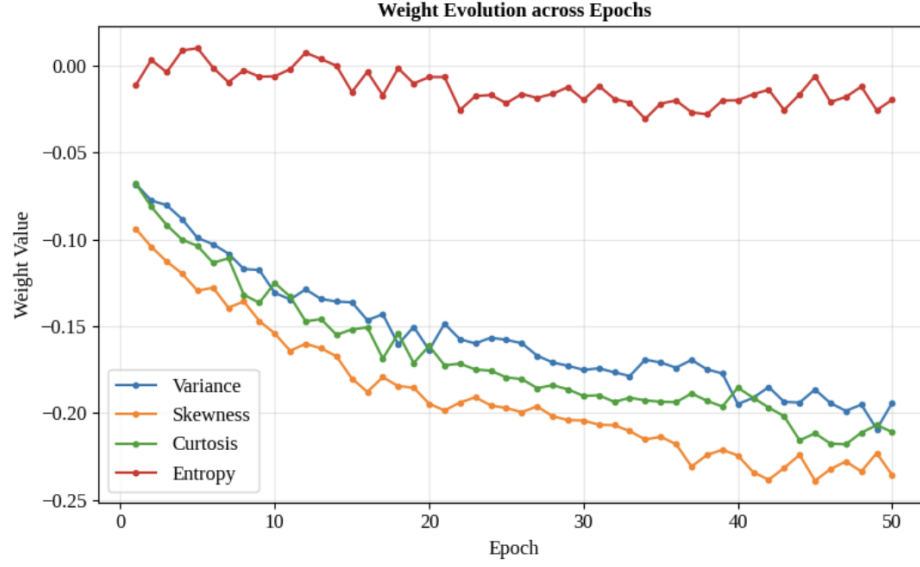


Figure 10: Weight Evolution across Epochs.

Interpretation: The weights for Variance, Skewness, and Kurtosis steadily grow in magnitude and stabilize as training progresses, showing the model learns meaningful, non-trivial weights for these features. The Entropy weight stays close to zero throughout, reinforcing that Entropy contributes minimally to the final decision boundary.

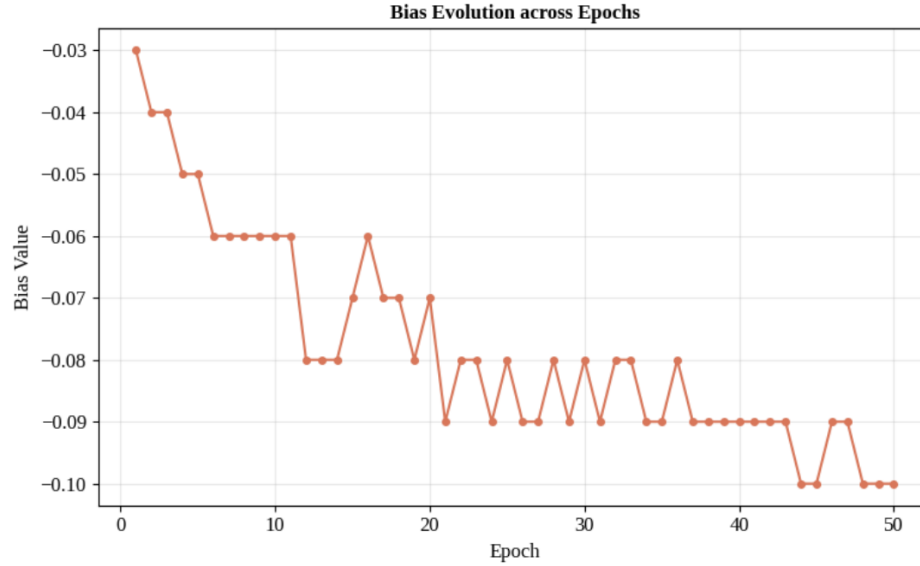


Figure 11: Bias Evolution across Epochs.

Interpretation: The bias term steadily decreases from zero to around -0.1 and stabilizes, indicating the decision boundary shifts to a consistent offset as training settles. The gradual, non-oscillatory change reflects stable learning.

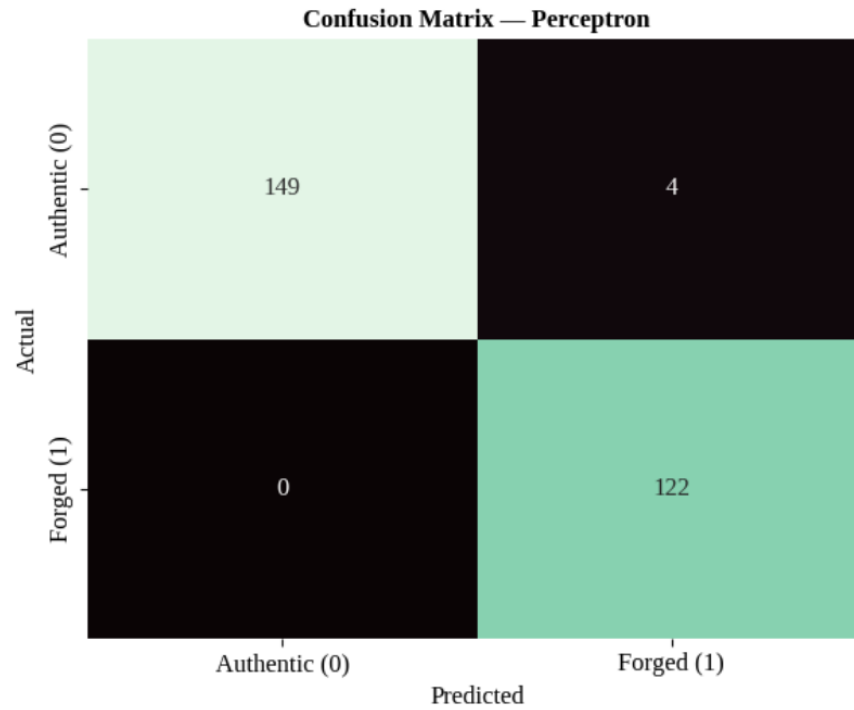


Figure 12: Confusion Matrix.

Interpretation: Out of 275 test samples, the model correctly identifies 149 authentic and 122 forged notes, with only 4 false positives and 0 false negatives. Every forged note is caught (recall = 1.0), and the very small number of errors confirms the model generalizes very well on this dataset.

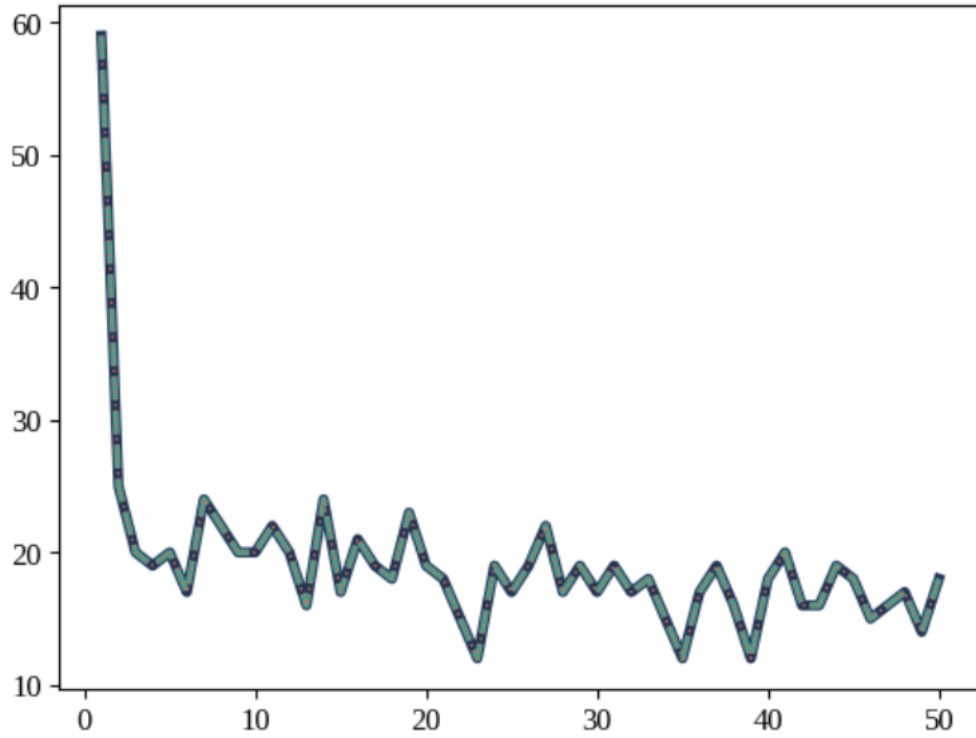


Figure 13: Learning Rate Comparison.

Interpretation: The error curves for the three learning rates follow essentially the same trajectory, and all reach the same test accuracy. This is expected: because the perceptron update depends only on the sign of the error ($y - \hat{y}$), the learning rate scales the weight magnitudes but not the direction of the decision boundary or the per-epoch error count.



Figure 14: Decision Boundary (Variance vs Skewness).

Interpretation: Using the two most discriminative features, the perceptron learns a single straight-line boundary that separates most of the two classes, with a small overlap region in the middle. This illustrates both the strength and the limitation of a linear classifier on near-separable data.

9. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80% / 20% (1097 / 275)
Learning Rate	0.01
Epochs	50
Initial Weights	[0, 0, 0, 0]
Initial Bias	0.0
Final Weights	[-0.1941, -0.2358, -0.2109, -0.0195]
Final Bias	-0.10
Accuracy	0.9855
Precision	0.9683
Recall	1.0000
F1-score	0.9839

Epoch-wise Learning

Epoch	Errors	w1 (Var)	w2 (Skew)	w3 (Cur)	Bias
1	59	-0.0680	-0.0938	-0.0678	-0.03
5	20	-0.0991	-0.1295	-0.1037	-0.05
10	20	-0.1307	-0.1541	-0.1250	-0.06
20	19	-0.1636	-0.1946	-0.1613	-0.07
30	17	-0.1751	-0.2043	-0.1901	-0.08
40	18	-0.1948	-0.2244	-0.1853	-0.09
50	18	-0.1941	-0.2358	-0.2109	-0.10

10. Discussion (Additional Tasks)

1. Compare the Step and Sigmoid activation functions.

Aspect	Step Function	Sigmoid Function
Output Range	Binary $\{0, 1\}$	Continuous $(0, 1)$
Differentiability	Non-differentiable at $z = 0$	Smoothly differentiable everywhere
Gradient	Zero almost everywhere	Provides a usable gradient
Output Type	Hard decisions	Probabilistic confidence values

Modern neural networks rely on gradient-based optimization and backpropagation, which require the derivative of the activation function. Because the step function has a derivative of zero everywhere, no gradient signal can flow backward to update earlier layers, so the network cannot learn. Sigmoid and other modern activations yield continuous, non-zero derivatives, enabling effective end-to-end gradient learning. This is why the step function is confined to the classical single-layer perceptron and is not used in modern deep networks.

2. Compare the implementation with Scikit-learn's Perceptron.

The from-scratch implementation and `sklearn.linear_model.Perceptron` share the same underlying learning rule, and their measured performance on the test set is very close:

Metric	From-Scratch	Scikit-learn
Accuracy	0.9855	0.9745
Precision	0.9683	0.9457
Recall	1.0000	1.0000
F1-score	0.9839	0.9721

Both models catch every forged note (recall = 1.0) and reach comparable accuracy. Scikit-learn additionally offers built-in regularization, early stopping, and optimized vector operations for large

datasets, whereas the from-scratch version gives full visibility into the epoch-wise error trajectory and parameter history. The small differences in the metrics are attributable to shuffling order and initialization defaults; both converge to equivalent linear decision boundaries.

3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.

All three learning rates produced the same test accuracy of 98.55%, and their per-epoch error counts followed essentially the same path. The only systematic difference was in the magnitude of the final weights, which scaled directly with the learning rate (the weights for $\eta = 0.1$ were roughly ten times those for $\eta = 0.01$). This is because the perceptron update depends only on the sign of the error ($y - \hat{y}$), so the learning rate scales the size of each update but not the direction of the boundary or the number of misclassifications per epoch. The perceptron is therefore largely insensitive to the choice of learning rate in this range.

4. Explain why a Single Layer Perceptron cannot solve the XOR problem.

A single-layer perceptron can only represent a linear decision boundary, since its output is a weighted sum passed through a step function. The XOR function is not linearly separable: its inputs $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 0$ place the two class-1 points on one diagonal and the two class-0 points on the other. No single straight line can separate these two diagonals, so a single perceptron can never classify all four points correctly. Solving XOR requires a non-linear boundary, which needs at least one hidden layer (a multi-layer perceptron).

5. Study the effect of feature normalization on model convergence.

Normalization has a positive effect on convergence. Without it, features with larger numeric ranges dominate the weight updates and the model learns unevenly and more slowly. With standardization (zero mean, unit variance), all four features contribute on a comparable scale, so the updates are balanced and the error curve stabilizes more smoothly and in fewer epochs. In this experiment the features were standardized before training, which contributed to the fast initial drop in training error observed in the training-error plot.

6. Implement the perceptron learning algorithm for the OR, NOT, and AND gates.

The perceptron learning rule was applied to the three linearly separable logic gates. The bias was absorbed as w_0 (an input of constant 1), weights were initialised to zero, and a learning rate of $\eta = 0.1$ was used. The activation uses $z \geq 0 \Rightarrow 1$, so a point lying exactly on the boundary is classified as 1. After every individual weight update, the updated weights and the corresponding decision boundary were recorded and plotted.

Gate	Truth Table	Weight Updates	Converged
OR	$(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 1$	5	Yes
AND	$(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 0$, $(1, 0) \rightarrow 0$, $(1, 1) \rightarrow 1$	8	Yes
NOT	$0 \rightarrow 1$, $1 \rightarrow 0$	2	Yes

Each gate is linearly separable, so the perceptron converges to zero misclassifications in a small number of updates. The OR and AND gates use two inputs and learn a slanted decision line, while the NOT gate has a single input (padded with a constant zero second input so the same 2D plotter applies) and learns a vertical threshold that separates input 0 from input 1.

OR Gate — Boundary after each update (5 updates)

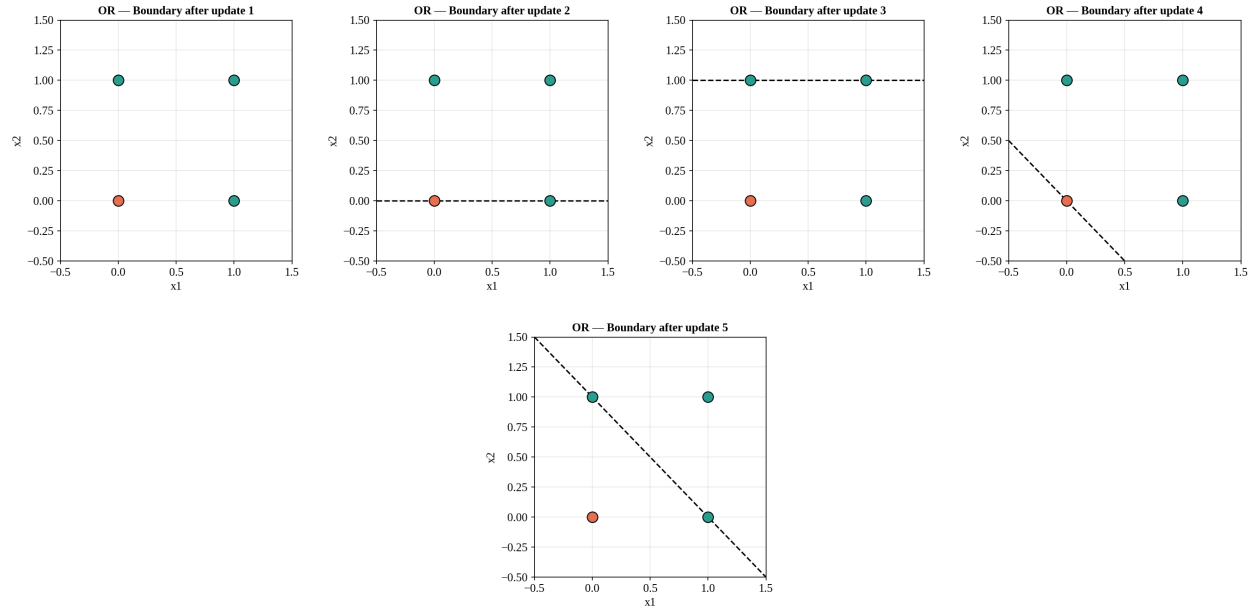


Figure 15: OR gate: decision boundary after each of the 5 weight updates.

AND Gate — Boundary after each update (8 updates)

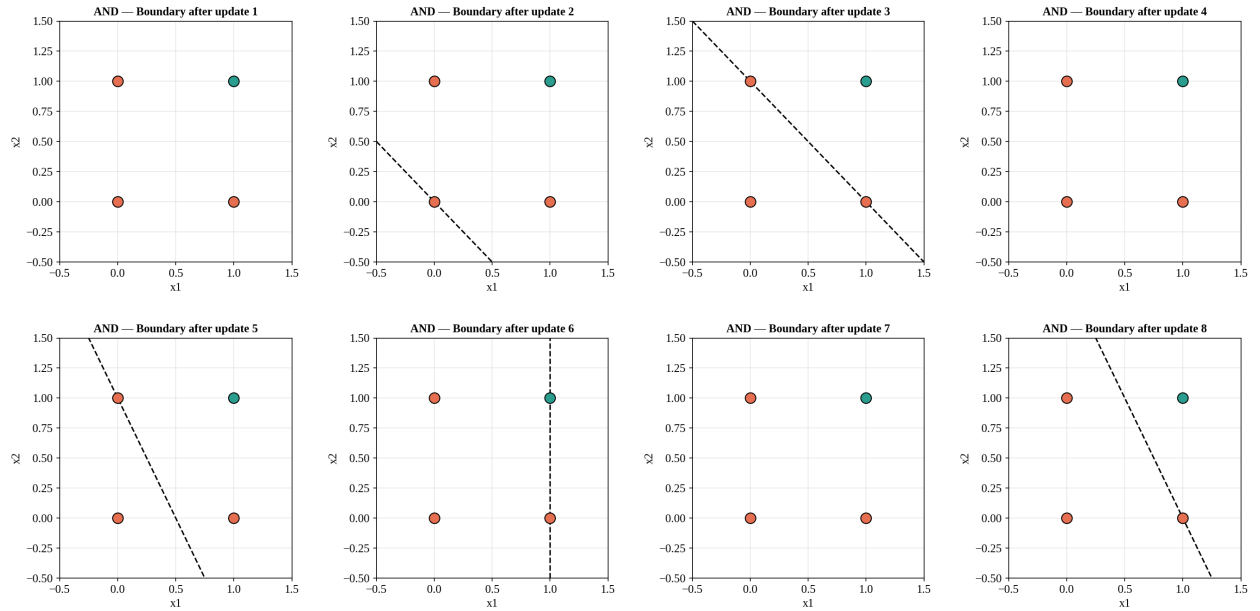


Figure 16: AND gate: decision boundary after each of the 8 weight updates.

NOT Gate — Boundary after each update (2 updates)



Figure 17: NOT gate: decision boundary after each of the 2 weight updates.

Interpretation: As each update shifts the weights, the decision line moves toward a separating position. Each saved plot shows the boundary *immediately after* a weight change, so an individual frame may look worse than the one before it: a single update corrects one misclassified point without regard to the others, and can temporarily push a previously-correct point to the wrong side. The final plotted frame therefore corresponds to the last correcting update, after which the perceptron completes a full error-free pass over the data (which triggers no further update, and hence no plot), confirming convergence. The AND gate requires the most updates (8), as its single positive corner (1, 1) lies closest to the boundary and demands the most repositioning; OR converges in 5 updates and NOT in 2. This behaviour contrasts with XOR (Task 4 above), which no single straight line can separate regardless of the number of updates.

11. Conclusion

A Single Layer Perceptron was implemented from scratch and trained on the Banknote Authentication dataset, achieving a test accuracy of 98.55% with perfect recall on the forged class. The training error dropped sharply and then plateaued at a small non-zero value, showing that the classes are nearly but not perfectly linearly separable. Variance and Skewness were the most discriminative features, while Curtosis and Entropy contributed less. The model matched Scikit-learn’s Perceptron closely and proved largely insensitive to the learning rate, consistent with the perceptron’s sign-based update rule. The experiment confirms the perceptron is well-suited to this near-separable task, while problems that are not linearly separable, such as XOR, would require a multi-layer network.

12. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.

5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.